

Akhil Yerrapally

Project Portfolio



B.S. Computer Engineering · University of Illinois Urbana-Champaign · Class of 2029

Seeking: Summer 2027 Internship – Embedded Systems, Controls, and Hardware

In this Portfolio

01: MPPT Solar Charge Controller

Power Electronics · PWM · DC-DC · Control · Telemetry

02: Self-Balancing Robot

ESP32 · C++ · PID Control · I2C · Sensor Fusion · Encoders

03: Intelligent Dog Door

Arduino · State Machines · C++ · SPI · Stepper Control

04: PCB Designs & Builds

PCB Game Console · Accumulator Control Module · Tools

05: Additional Builds

Light-seeking Car · Resistor-key Circuit

Skills & Domain Expertise

Languages/Software: C, C++, Python, Git, Linux, KiCAD, Altium Designer, Fusion 360

Embedded: ESP32, Arduino, Raspberry Pi, CAN, I2C Devices, SPI, UART, PWM, ADC, hardware interrupts / ISRs, quadrature decoding, non-blocking state machines

Controls: PID tuning, complementary filtering, IMU sensor fusion, stepper motor control, cascade loop architecture, analog filtering, MPPT algorithms, closed-loop regulation

Hardware: Motor drivers, RFID, voltage/current sensing, DC-DC converters, soldering, breadboard prototyping, decoupling & signal integrity, multimeter, oscilloscope

Contact Information

akhilyerrapally@gmail.com · (224) 469-7572

github.com/Ak-yerr · linkedin.com/in/Akhil-Yerrapally

MPPT Solar Charge Controller

A custom buck-converter-based charge controller that dynamically tracks maximum power point using perturb-and-observe logic.

My Role: Solo Build

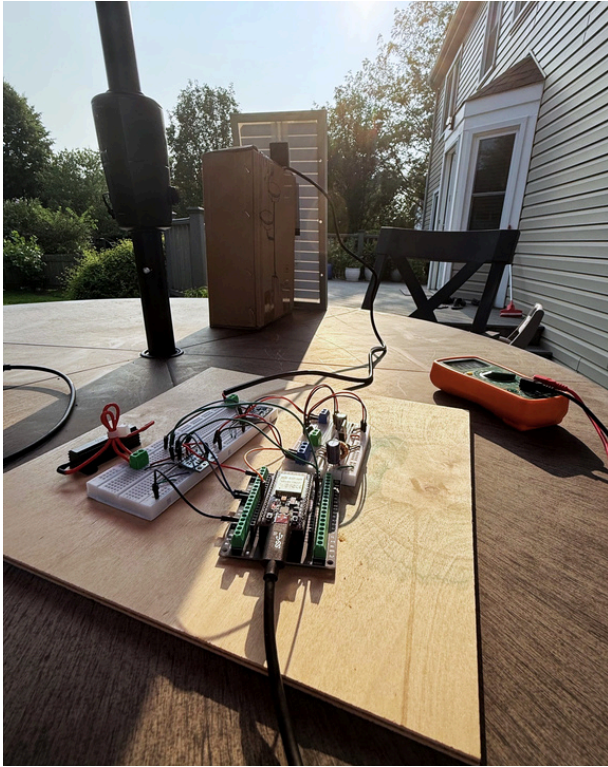


FIG. 1: Final prototype with MPPT charge control: closed-loop power tracking with an asynchronous, custom-built DC-DC converter

WHAT IT DOES

- Two INA226 monitors read voltage & current on the panel and load sides, feeding live measurements to an ESP32 over I2C
- A perturb-&-observe loop taps the PWM duty cycle: it watches and steps towards whichever direction raises power output
- A custom buck converter lowers the panel voltage while a level shifter allows the 3.3 V ESP32 to safely control the power switch

DESIGN & PROTOTYPE

- **Topology & impedance matching:** An asynchronous buck tracks $load \div D^2$, so sweeping duty sweeps the apparent load and gives the tracker a continuous peak-power output pointer
- **Gate drive:** An NPN transistor pulls the P-MOS gate below the panel rail while two series zeners clamp gate-to-source voltage within the ± 20 V limit as overload protection
- **Measurement & control:** Two INA226 monitors bracket the converter on independent I2C buses, so the panel-side reading drives the P&O loop while both together give live efficiency
- **Bring-up debugging:** Performed staged power-stage bring-up to isolate control vs. power faults; verified PWM switching and monitored gate-to-source voltage waveforms to prevent shoot-through before applying high supply voltage

RESULTS

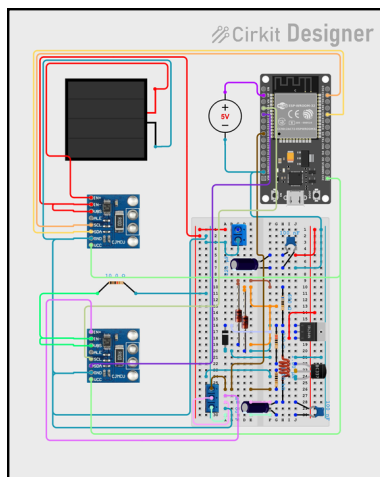


FIG. 2: Wiring diagram: Async. custom buck, sensors, & control

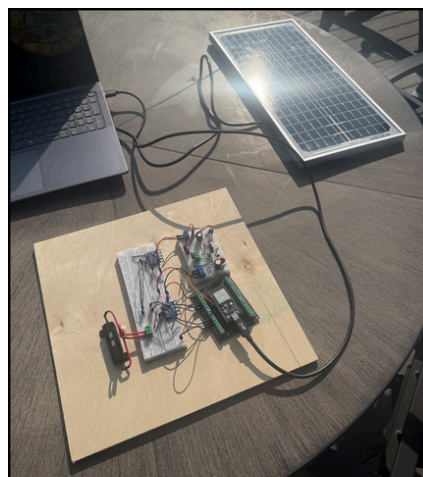


FIG. 3: As-built extracted circuit and measurement tools required

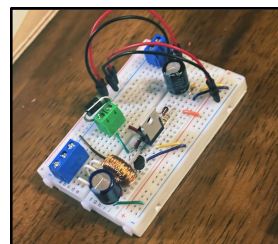
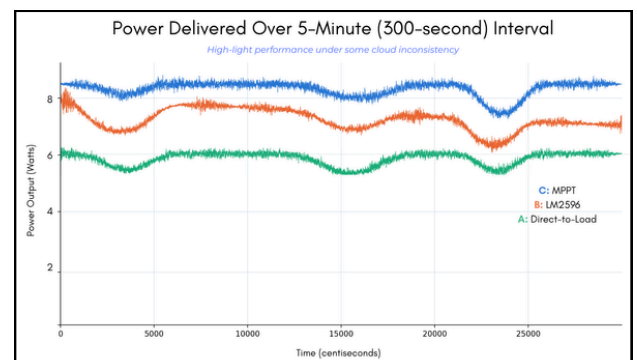


FIG. 4: Custom async. DC-DC buck converter

Testing achieved a **15% power delivery increase** over a commercial buck converter & **42% increase** over a direct-to-load (33Ω Power Load) system in dynamic solar conditions



- Stepping duty across its range locates the true peak, then P&O continuously converges on that same point

Self-Balancing Robot

An inverted-pendulum robot that holds itself upright on hardwood, carpet, and bumpy terrain, and recovers from a physical shove.

My Role: Solo Build

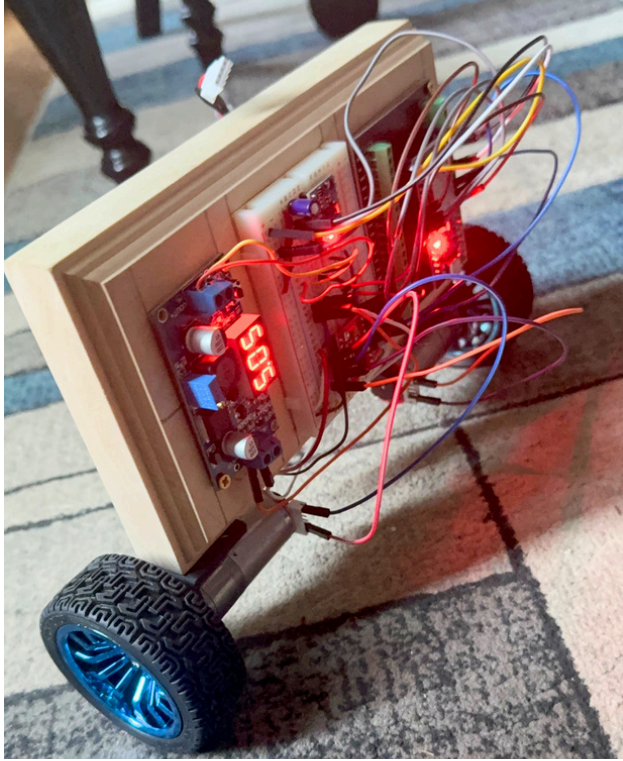


FIG. 1: Final prototype (P4) actively balancing: single PID loop for control logic, 3S LiPo being the central, rechargeable power source

WHAT IT DOES

- MPU-6050 IMU feeds a complementary filter that fuses gyro & accelerometer data into a pitch estimate
- A PID loop compares that pitch against a calibrated upright setpoint and computes a correction
- A TB6612FNG drives two encoder gear motors to hold the robot upright and recover from disturbances

DESIGN & PROTOTYPE

- **Design pivot:** Four chassis iterations – the fix was raising the center of mass. Prototypes 2 and 3 sat too low to absorb terrain without oscillating out of control
- **Control law:** Single PID loop on pitch error with an added cubic term, so correction scales up as tilt grows instead of responding uniformly. Encoder RPM decoded in hardware ISRs at 4x quadrature resolution
- **Angle estimation:** Complementary filter at $\alpha = 0.98$, IMU registers read directly over I2C. Mounted centered over the axle so movement doesn't corrupt the estimate
- **Power & bring-up:** 3S LiPo to the driver, buck-converted to 5V for the ESP32, common ground. Motor B originally on GPIO12 (strapping pin), broke flashing until remapped

RESULTS

Balancing: 100% success (3 settled indefinitely, with the remaining 2 tests lasting over 100 seconds)

Terrain: Final prototype correctly balanced on flat hardwood & uneven carpet

Recovery: 3/5 tap trials recovered within 5 seconds of the disturbance; other 2 trials oscillated & gave out

Bottleneck: Motor torque and overall chassis rigidity

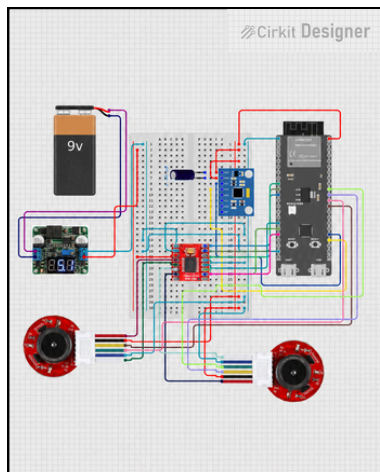


FIG. 2: Wiring diagram: Logic, sensors, & power distribution

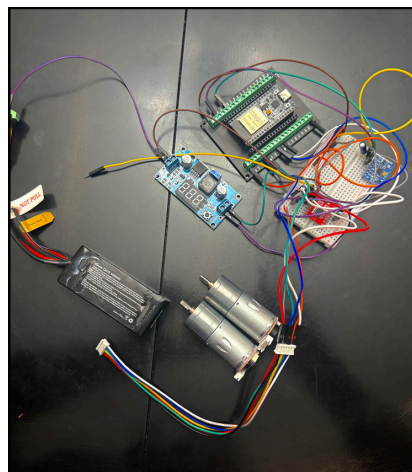


FIG. 3: As-built circuit (encoders & temp. 2S LiPo) without chassis

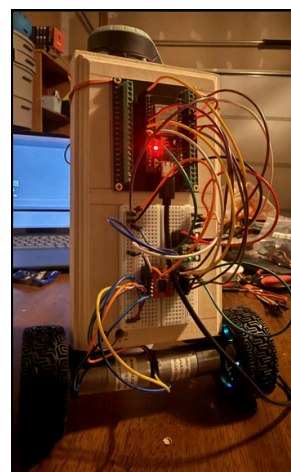


FIG. 4: Early prototype; failed recovery tests

- A cascade variant (double-loop) is also running on the same hardware: an outer angle loop setting a target wheel speed, with an independent inner RPM loop per motor

Intelligent Dog Door

An RFID-authenticated pet entry that opens only for a tagged animal and mechanically locks itself shut against anything else.

My Role: Software and Electronics Lead



FIG. 1: Installed prototype: worm drive at left, door in the open position with both sensors activated

FIG. 2: Installed prototype: Idle position, door closed

WHAT IT DOES

- A PIR motion sensor wakes the system when an animal approaches the door
- An RC522 reader authenticates the pet's collar tag over SPI; unknown tags are ignored
- A NEMA 17 stepper swings the door open, then closes it into a self-locking gear state

DESIGN & PROTOTYPE

- **Design pivot:** BLE/actuators replaced with RFID & worm gear – offering higher ID resolution, automatic locking
- **Firmware:** Non-blocking C++ state machine – *AccelStepper* commands the A4988 in open-loop step counts so the main loop keeps polling sensors instead of stalling inside *delay()*
- **Power & signal integrity:** 5V logic interfaced to a 3.3V SPI peripheral, with the motor on a separate 9V rail and a 100 μ F bulk cap to keep driver switching transients off the logic supply. Set coil current by tuning the A4988 V_{ref}
- **Mechanism:** A worm-to-spur gear train is non-backdrivable, so the closed door resists being forced open with the motor unpowered

RESULTS

92% final product reliability score after administering various functionality tests

Final prototype withstands harsher weather conditions

Live Data: The program accurately logs entry and exit times on the Arduino's local web server

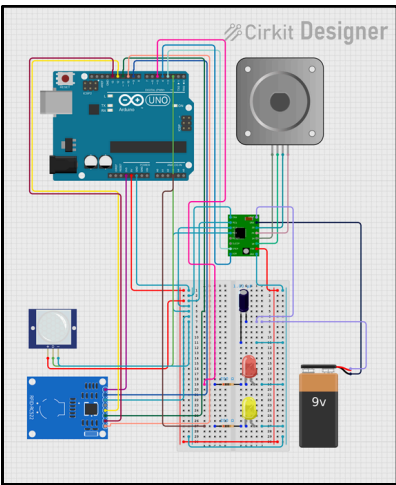


FIG. 3: Wiring diagram: Logic, driver, and 9V motor power rail

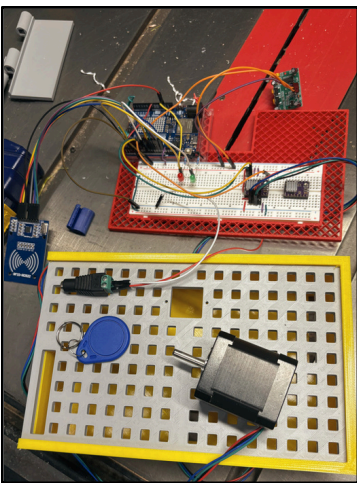


FIG. 4: As-built central circuit with vented enclosure panel

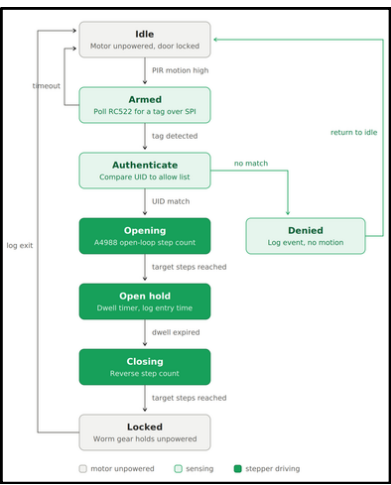


FIG. 5: Dog door prototype's state machine flowchart

- Built by a 4-person team over 6 months for the Project Lead the Way (PLTW) capstone project
- Final prototype ran reliably through demo-day testing on passive cooling via the vented enclosure

PCB Designs & Builds

SIMPLE GAME CONSOLE

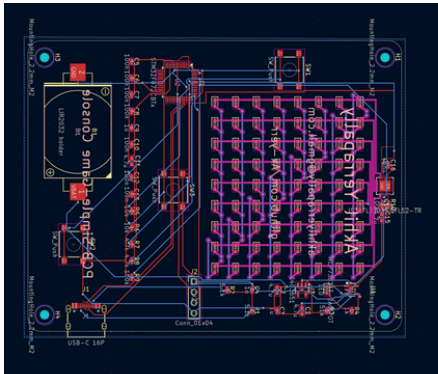


FIG. 1: Wiring Schematic: Utilized KiCAD to engineer the schematic for power block with coin cell + USBC charger, STM 32 MCU module with manual switches for gameplay/reset controls, and LED matrix output stage with test point (routed on KiCAD)

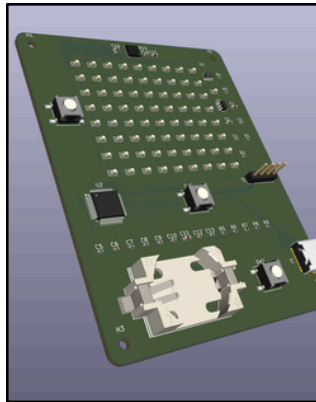


FIG. 2: PCB Routing on KiCAD: The final 3D board layout with switch placements, 2-layer design, and coin cell charging section

DESIGN & PROTOTYPE

- **Board Design:** Small-sized PCB to run simple 1/2 button games (i.e. Flappy Bird, Cube Field)
- **Custom 9x16 charlieplex LED matrix:** 72 LEDs in a compact matrix driven off just 9 outputs of an IS31FL3731, with routing built around each net's odd vertical-plus-split-row shape instead of a standard grid
- **Crystal-less USB on an STM32F072:** The internal oscillator is trimmed against USB timing packets instead of using an external crystal, with a hidden button combo enabling firmware updates over USB with no external programmer (hold S2, press S3 to flash)
- **Battery-safe charging by design:** Using MCP73831 because coin cell's low capacity can't handle TP4056's min. charge current

ACCUMULATOR CONTROL MODULE

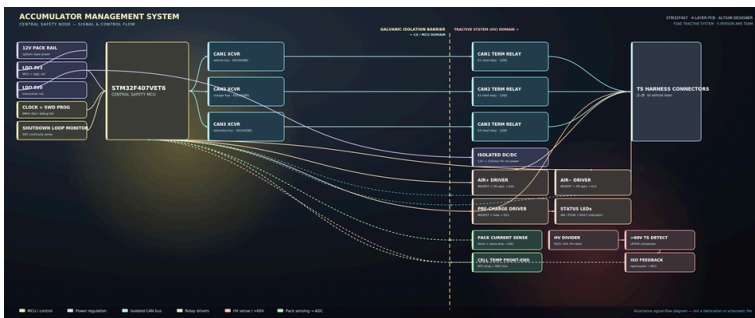


FIG. 3: Flowchart of accumulator safety-node PCB, showing STM32 MCU connected to color-coded blocks across an isolation barrier



FIG. 4: Accumulator management system installed on the car; cell supervision circuit, AMS PCB, and the precharge circuit PCB constitute the three different stages/parts

DESIGN & PROTOTYPE

- **Board Design:** A 4-layer PCB built around the STM32F407VET6 MCU in Altium Designer, serving as the central safety node of the FSAE tractive system; meeting Formula SAE electrical rulebook requirements
- **CAN Architecture:** Three isolated CAN buses (vehicle, charger, telemetry) with 120Ω reed-relay switched termination, with correct bus impedance in running & charging configs.
- **Relay Control:** MOSFET driver and feedback circuits for the AIR+/AIR- and precharge relays, with a pack current sense → HV divider → comparator chain to detect Tractive System Active at 60VDC threshold
- **Team Role:** Worked with 5-person AMS team on the control module specifically; contributed PCB routing in Altium Designer and supported its installation into AMS

Additional Builds

Wheatstone Bridge • Voltage Dividers • Comparators • MOSFET Switching • Discrete Logic • RC Timing • Photoresistive Sensing • Oscilloscope Debugging

RESISTOR-KEY LOGIC CIRCUIT

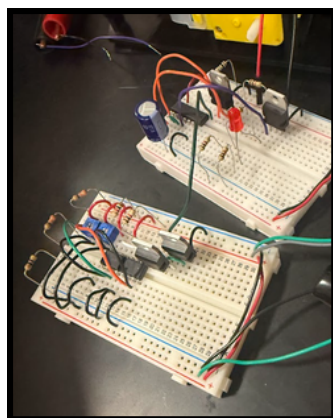
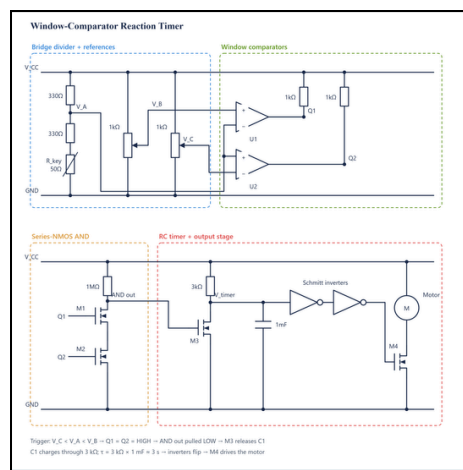


FIG. 2: As-built resistor-key system: Breadboard setup with potentiometer tuning and timed MOSFET motor gate drive

DESIGN & PROTOTYPE

- **Detection window:** A Wheatstone bridge subcircuit puts V_A at 4.81 V with the correct $50\ \Omega$ key; two pots set comparator references 60 mV apart. Wrong keys miss it, with $0\ \Omega$ dropping V_A to 4.32 V and $1\ \text{k}\Omega$ pushing it to 7.13 V
- **Discrete logic:** The AND stage is two series N-channel MOSFETs rather than a logic IC, measuring 8.9 V HIGH and 0.1 V LOW
- **Anti-brute-force timing:** A $3\ \text{k}\Omega / 1000\ \mu\text{F}$ RC ($\tau = 3\ \text{s}$) gates the motor, with a shunt nMOS dumping the cap the instant the AND output drops. Sweeping a pot never accumulates charge, so the key must hold correct for 3 s

LIGHT-SEEKING VEHICLE

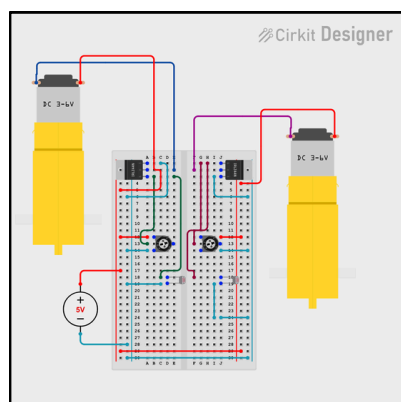


FIG. 3: Wiring diagram: Breadboard layout and dual-channel nMOSFET speed control circuit for light-seeking behavior

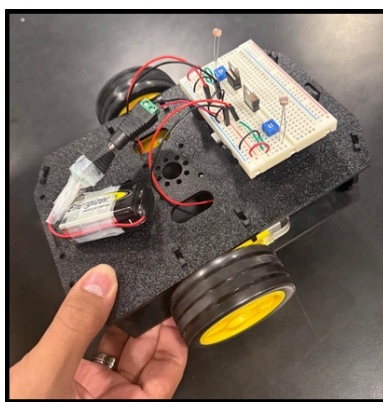


FIG. 4: As-built prototype: Breadboard circuit, sensor roles, dual DC motor assembly, working under high-light

DESIGN & PROTOTYPE

- **Analog speed control:** A photoresistor / potentiometer divider drives each nMOSFET gate directly, so motor speed scales continuously with the environment's brightness rather than simply switching at a threshold
- **Channel matching:** Sensitivity mismatch between the two channels translated into steering bias, so the early build just spun in place. Pots required tuning to confirm both gates tracked on the scope
- **Discrete logic:** Turning, stopping, and variable speed all emerge from component placement. The control logic is based on the resistor that sits on top of the divider